

GenAI Application Development: Phase 5

Study Guide: Safety, Evaluation & Monitoring on Databricks

1. Implement LLM guardrails to prevent negative outcomes

Source: <https://docs.databricks.com/en/machine-learning/model-serving/guardrails.html>

Core Concept: AI Guardrails intercept both user inputs and LLM outputs to filter out harmful, toxic, or personally identifiable information (PII) before it reaches the model or the end-user. Databricks provides this natively at the Model Serving layer.

How it works on Databricks: When configuring a Databricks Model Serving endpoint (either a Foundation Model or a custom agent), you can enable built-in safety filters. These filters automatically block restricted content (e.g., hate speech, violence) and can optionally mask PII. If an input violates the guardrail, the endpoint returns a blocked response message instead of passing the prompt to the LLM.

When to use vs. alternatives:

- **Use Databricks AI Guardrails:** For production endpoints to ensure baseline safety and compliance without writing custom regex or relying solely on system prompts (which can be jailbroken).
- **Alternatives:** System prompt engineering (less reliable, easily bypassed via jailbreaks) or chaining a secondary "Moderation LLM" (adds latency and cost).

Common Mistakes: Assuming system prompts ("Do not output toxic content") are sufficient guardrails. Malicious users can use prompt injection to bypass these instructions, making infrastructure-level guardrails critical.

Code Example & Function: Enabling guardrails on a serving endpoint using the Databricks SDK.

```

from databricks.sdk import WorkspaceClient
from databricks.sdk.service.serving import AiGuardrails, PiiBehavior

w = WorkspaceClient()

# Function: w.serving_endpoints.create() / update_config()
# Configures the serving endpoint to natively apply safety filters
# and PII masking before fulfilling requests.
w.serving_endpoints.update_config(
    name="my-chat-endpoint",
    served_entities=[...], # Existing model config
    ai_guardrails=AiGuardrails(
        pii_behavior=PiiBehavior.MASK,
        profanity_filter=True,
        hate_speech_filter=True
    )
)

```

2. Qualitatively assess responses to identify common issues (quality & safety)

Source: <https://docs.databricks.com/en/generative-ai/agent-evaluation/index.html>

Core Concept: Qualitative assessment involves judging the subjective quality of an LLM's response (e.g., "Is this helpful?" or "Is this safe?"). Mosaic AI Agent Evaluation provides frameworks to measure this using both LLM-as-a-judge (automated) and human-in-the-loop (manual) feedback.

How it works on Databricks: Databricks utilizes `mlflow.genai.evaluate` equipped with built-in LLM judges. You provide an evaluation dataset (prompts and ground-truth answers), and Databricks uses a hosted LLM judge to score the output for metrics like `AnswerCorrectness`, `Relevance`, and `Safety`. Additionally, Databricks provides a "Review App" UI where human domain experts can read agent traces and give thumbs-up/down feedback to calibrate the automated judges.

When to use vs. alternatives:

- **Use Mosaic AI Agent Evaluation:** During CI/CD pipelines to automatically benchmark a new model version against a golden dataset before deployment.
- **Alternatives:** Manual qualitative testing by developers. This is unscalable, subjective, and difficult to track across iterations compared to automated MLflow metrics.

Common Mistakes: Deploying agents based solely on quantitative metrics (like latency or exact-match accuracy) without measuring semantic relevance or safety. An LLM can confidently output an incorrect or toxic response rapidly, which exact-match metrics might miss.

Code Example & Function: Running qualitative safety and relevance checks.

```
import mlflow
from mlflow.genai.scorers import AnswerCorrectness, Safety

# Function: mlflow.genai.evaluate()
# Uses an LLM-as-a-judge to qualitatively grade the outputs against
# safety guidelines and ground-truth accuracy.
results = mlflow.genai.evaluate(
    model="endpoints:/my-chat-endpoint",
    data=eval_dataset_pandas_df,
    targets="ground_truth",
    scorers=[
        AnswerCorrectness(),
        Safety() # Evaluates for toxicity, violence, etc.
    ]
)
print(results.metrics_summary)
```

3. Compare Evaluation and Monitoring phases of the GenAI life cycle

Source: <https://docs.databricks.com/en/machine-learning/model-serving/inference-tables.html> & <https://docs.databricks.com/en/lakehouse-monitoring/index.html>

Core Concept:

- **Evaluation (Pre-deployment):** Assesses model performance *offline* against a static, curated "golden" dataset using MLflow. The goal is to ensure the model meets quality/safety thresholds before release.
- **Monitoring (Post-deployment):** Tracks the model's performance *online* in real-time against live user traffic. The goal is to detect data drift, increased error rates, or toxic user inputs over time.

How it works on Databricks: While Evaluation relies on MLflow, Monitoring relies on **Inference Tables** and **Databricks Lakehouse Monitoring**. When Inference Tables are enabled on a Model Serving endpoint, Databricks automatically logs every incoming request and outgoing response to a Delta table in Unity Catalog. Lakehouse Monitoring can then be scheduled to analyze this table, generating statistical profiles and dashboards to track token usage, latency, and response anomalies over time.

When to use vs. alternatives:

- **Use Both:** They are complementary. Evaluation acts as the gatekeeper for CI/CD; Monitoring acts as the ongoing health check in production.
- **Alternatives:** Building custom logging logic inside your agent application to write API calls to a database. Databricks Inference Tables do this natively at the infrastructure level with zero application-code changes.

Common Mistakes: Failing to enable Inference Tables on a production endpoint. Without them, you have no historical record of what users are asking the LLM or how it is responding, making it impossible to audit safety incidents or identify performance drift.

Code Example & Function: Enabling Inference Tables for ongoing monitoring.

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.serving import auto_capture_config

w = WorkspaceClient()

# Function: auto_capture_config()
# Configures the endpoint to automatically log all requests and responses
# to a Unity Catalog Delta table for continuous Lakehouse Monitoring.
config = auto_capture_config(
    catalog_name="main",
    schema_name="monitoring",
    table_name_prefix="agent_endpoint_logs",
    enabled=True
)

w.serving_endpoints.update_config(
    name="my-chat-endpoint",
    served_entities=[...],
    auto_capture_config=config
)
```